

Lab1

Step 1 – Install Maven (Windows) Steps

Download latest binary ZIP from [Maven Download Page](#)

Extract ZIP and move folder to C:\Program Files\apache-maven-x.x.x

Open System Environment Variables → Path → New

Paste: C:\Program Files\apache-maven-x.x.x\bin

Click OK and save

Verify Maven Installation CMD

```
mvn -v
```

Step 2 – Install Gradle (Windows) Steps

Download latest binary ZIP from [Gradle Downloads Page](#)

Extract ZIP and move folder to C:\Program Files\gradle-x.x.x

Open System Environment Variables → Path → New

Paste: C:\Program Files\gradle-x.x.x\bin

Click OK and save

Verify Gradle Installation CMD

```
gradle -v
```

Lab2

Step 1 – Create Project Folder & Maven Project CMD

```
mkdir program2
```

```
cd program2
```

```
mvn archetype:generate -DgroupId=com.example -DartifactId=myapp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

Step 2 – Navigate into Project Folder CMD

```
cd myapp
```

```
notepad pom.xml
```

Step 3 – editpom.xml

add the below code before </project>

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>2.22.2</version>
    </plugin>
  </plugins>
</build>
</project>
```

Step 4 – Compile the Project CMD

```
mvn compile
```

Step 5 – Run Unit Tests CMD

```
mvn test
```

Step 6 – Package into JAR CMD

```
mvn package
```

Step 7– Run the Application CMD

```
java -cp target/myapp-1.0-SNAPSHOT.jar com.example.App
```

Expected Output = “helo world”

Lab3

```
C:\Users\lliki>mkdir program3-groovy
```

```
C:\Users\lliki>cd program3-groovy
```

```
C:\Users\lliki\program3-groovy>gradle init
```

Select type of build to generate:

1: Application

Enter selection (default: Application) [1..4] 1

Select implementation language:

1: Java

Enter selection (default: Java) [1..6] 1

Enter target Java version (min: 7, default: 21): 17

Project name (default: program3-groovy): groovyProject

Select application structure:

1: Single application project

2: Application and library project

Enter selection (default: Single application project) [1..2] 1

Select build script DSL:

1: Kotlin

2: Groovy

Enter selection (default: Kotlin) [1..2] 2

Select test framework:

1: JUnit 4

2: TestNG

3: Spock

4: JUnit Jupiter

Enter selection (default: JUnit Jupiter) [1..4] 1

Generate build using new APIs and behavior (some features may change in the next minor release)? (default: no) [yes, no] no

> Task :init

BUILD SUCCESSFUL in 45s

1 actionable task: 1 executed

C:\Users\lliki\program3-groovy>gradle run

> Task :app:run

Hello World!

BUILD SUCCESSFUL

C:\Users\lliki\program3-groovy>gradle test

Lab5

Step 1 – Prerequisites

java -version

Step 2 – Install Jenkins on Windows

Download Jenkins Windows installer from jenkins.io

Run the installer

Select “Run service as LocalSystem”

Set Port: 8080 (or custom port like 3030)

Click Test → Next

Select Java Directory:

C:\Program Files\Java\jdk-21\

Click Next

Allow permissions if prompted

Click Finish to complete installation

Step 3 – First-Time Jenkins Setup

Open Browser: <http://localhost:8080>

Copy password from: C:\ProgramData\Jenkins\jenkins\secrets\initialAdminPassword

Paste the password in Jenkins setup page

Click Continue , Click Install suggested plugins

Wait for plugin installation

Create Admin User

Click Save and Continue ->Click Save and Finish ->Click Start using Jenkins

Result

Jenkins Dashboard opens successfully with options:

New Item

Build History

Manage Jenkins

My Views

lab4

Step 1 – Create Maven Project CMD - mkdir program4 -- cd program4

```
mvn archetype:generate -DgroupId=com.example -DartifactId=maven-example -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

cd maven-example

Step 2 – Open pom.xml CMD

notepad pom.xml

Step 3 – paste before </project>

```
<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
</plugin>
</plugins>
</build>
</project>
```

Step 4 – Replace App.java

Path: src/main/java/com/example/App.java

package com.example;

```
public class App {
    public static void main(String[] args) {
        System.out.println("Hello, Maven");
        System.out.println("This is the simple realworld example ....");
        int a = 5;
        int b = 10;
        System.out.println("Sum of " + a + " and " + b + " is " + sum(a, b));
    }
    public static int sum(int x, int y) {
        return x + y;
    }
}
```

Step 5 – Build & Run with Maven CMD

mvn clean install

mvn exec:java -Dexec.mainClass="com.example.App"

Output

Hello, Maven

This is the simple realworld example

Sum of 5 and 10 is 15

Step 6 – Initialize Gradle in Same Folder CMD

gradle init

When prompted:

Generate Gradle build from Maven? → yes

DSL → 2 (Groovy)

New APIs → no

Step 7 – Replace build.gradle

```
plugins {
    id 'java'
}
group = 'com.example'
version = '1.0-SNAPSHOT'
repositories {
    mavenCentral()
}
dependencies {
    testImplementation 'junit:junit:4.12'
}
task run(type: JavaExec) {
    main = 'com.example.App'
    classpath = sourceSets.main.runtimeClasspath
}
```

Step 8 – Build & Run with Gradle CMD

gradlew build - gradlew run

Lab 6 Step 1 – Install Plugins

Manage Jenkins → Manage Plugins → Available Plugins

Install: Maven Integration Plugin

Install: Gradle Plugin

Install: Git Plugin

Manage Jenkins → Global Tool Configuration

Add Maven Installation

Add Gradle Installation

Step 3 – Create Job

New Item → Enter Job Name → Freestyle Project

Source Code Management → Git → Enter Repository URL

Step 4 – Configure Trigger

Build Triggers → Poll SCM → H/5 * * * *

Step 5 – Add Build Steps

Build → Invoke top-level Maven targets → clean install

Build → Invoke Gradle script → clean build

Step 6 – Post Build

Post-build Actions → Publish JUnit test result report → **/TEST-*.xml

Step 7 – Run Build

Save → Build Now

Open Build Number → Console Output

Lab7

Step 1 – Install Ansible on Windows - > Python must be installed. - >Verify Installation CMD - >python --version

Step 2 – Install Ansible - >Open CMD

pip install ansible

ansible --version

Step 3 – Create Inventory File

Create inventory.ini

[windows]

server1 ansible_host=192.168.1.10

server2 ansible_host=192.168.1.11

Step 4 – Create Playbook

Create basic_playbook.yml

name: Configure Windows Servers

hosts: windows

tasks:

name: Create Folder

win_file:

path: C:\Temp\Test

state: directory

name: Copy File

win_copy:

src: ./sample.txt

dest: C:\Temp\Test\sample.txt

name: Run Command

win_command: ipconfig

Step 5 – Run Playbook CMD

ansible-playbook -i inventory.ini basic_playbook.yml

Step 6 – Verify Configuration CMD

dir C:\Temp\Test

Result

Folder and file are created successfully on Windows server.

Lab8

Step 1 – Create Freestyle Jenkins Job Steps

New Item → Freestyle Project

Source Code Management → Git → Enter Repo URL

Build Step → Invoke top-level Maven targets

Goals: clean package
Post-Build Actions → Archive the artifacts
Files: target/*.jar
Save → Build Now
Verify Build CMD
dir target

Step 2 – Install Ansible (Windows) CMD

```
python --version  
pip install ansible  
ansible --version
```

Step 3 – Create inventory.ini

```
[target_servers]  
server1 ansible_host=192.168.1.10  
server2 ansible_host=192.168.1.11
```

Step 4 – Create deploy_playbook.yml

```
- name: Deploy Maven Artifact  
  hosts: target_servers  
  tasks:  
    - name: Copy artifact  
      win_copy:  
        src: C:\Jenkins\workspace\MyJob\target\artifact.jar  
        dest: C:\Temp\artifact.jar  
    - name: Run Application  
      win_command: java -jar C:\Temp\artifact.jar
```

Step 5 – Add Ansible to Jenkins Job Steps

Manage Jenkins → Manage Plugins → Install Ansible Plugin
Jenkins Job → Add Build Step → Execute Windows batch command

Step 6 – Execute Batch Command CMD

```
ansible-playbook -i inventory.ini deploy_playbook.yml
```

Step 7 – Save & Build Steps

Save → Build Now → Monitor Console Output

Result

Maven project builds successfully and Ansible deploys the .jar file to Windows servers.

Lab9----- Step 1 – Create Azure DevOps Account STEPS Visit dev.azure.com Sign in with Microsoft account or create new account Step 2 – Create Organization STEPS After sign-in create new organization Enter organization name select region complete setup Step 3 – Create New Project STEPS Click New Project Project Name enter unique name Description optional Visibility Public or Private click Create Step 4 – Explore Project Dashboard STEPS Open Boards Repos Pipelines Artifacts Add work items in Boards Create repositories in Repos Set up CI/CD in Pipelines

Lab10

Step 1 – Integrate GitHub Repository STEPS

Go to Project Settings → Service connections → New service connection → GitHub → authenticate & authorize
Go to Pipelines → Create Pipeline → select GitHub repo

Step 2 – Azure Repos (alternative) STEPS

During pipeline creation select Azure Repos Git

Step 3 – Azure Pipeline YAML for Maven

trigger - main

pool vmImage: ubuntu-latest

steps

MavenAuthenticate@0 mavenServiceConnection: your-service-connection

Maven@3 mavenPomFile: pom.xml goals: clean package options: -DskipTests=false publishJUnitResults: true

testResultsFiles: **/TEST-.xml codeCoverageToolOption: jacoco

PublishTestResults@2 testResultsFormat: JUnit testResultsFiles: **/TEST-.xml failOnBadExitCode: true

Step 4 – Azure Pipeline YAML for Gradle

trigger - main

pool vmImage: ubuntu-latest

steps

Gradle@2 workingDirectory: \$(System.DefaultWorkingDirectory) gradleWrapperFile: gradlew options: build

publishJUnitResults: true testResultsFiles: /build/test-results//.xml

PublishTestResults@2 testResultsFormat: JUnit testResultsFiles: /build/test-results//.xml failOnBadExitCode: true

Step 5 – Save & Run Pipeline STEPS

Save YAML file → pipeline auto-triggers or Run Pipeline

Step 6 – View Results STEPS

Pipelines → select run → view logs, test results, code coverage

Lab11

Step 1 – Create Azure App Service STEPS

Azure Portal → Create a resource → Web + Mobile → Web App

Fill Subscription, Resource Group, Name, Runtime Stack → Review + Create

Step 2 – Create Azure Key Vault STEPS

Azure Portal → Create a resource → Security + Identity → Key Vault

Create vault → Go to Secrets → Add connection strings / API keys

Access policies → add Azure DevOps service principal

Step 3 – Create Release Pipeline STEPS

Azure DevOps → Pipelines → Releases → New pipeline

Choose Empty Job → add stage (Production) → configure

Step 4 – Azure App Service Deploy Task STEPS

Stage → + → Azure App Service Deploy

Select subscription → App Service name → package path \$(System.DefaultWorkingDirectory)/drop//*.*.zip**

Step 5 – Azure Key Vault Task STEPS

Stage → + → Azure Key Vault task

Fetch secrets → map as environment variables

Step 6 – Enable Continuous Deployment STEPS

Click Triggers → enable Continuous deployment

Link build pipeline → Save → Create Release

Step 7 – Monitor & Rollback STEPS

Releases → view logs/status → if failure click Redeploy for rollback

Lab12

Step 1 – Define Application STEPS

git init , git add . ,git commit -m "Initial commit" ,git remote add origin <repo-url> ,git push -u originmain

Step 2 – Azure DevOps Setup STEPS

Create account at dev.azure.com -- > Create New Project

Step 3 – Build Pipeline YAML

trigger: main

pool: ubuntu-latest

steps

NodeTool@0 versionSpec: 18.x

npm install

npm test

npm run build

PublishBuildArtifacts@1 PathToPublish: \$(Build.ArtifactStagingDirectory) ArtifactName: drop

PublishTestResults@2 testResultsFormat: JUnit testResultsFiles: **/test-results/*.xml

Step 4 – Release Pipeline STEPS

Releases → New Pipeline

Stages: Dev → Staging → Production

Add Azure App Service Deploy task

Enable Continuous DeploymentStep 5 – Azure Key Vault STEPS

Create Key Vault → add secrets (API keys, connection strings)

Release Pipeline → Add Key Vault taskStep 6 – Testing & Monitoring STEPS

Add unit tests in build

Use Selenium / Postman for E2E testing

Enable Application Insights